

Лабораторная №2 Работа с массивами

Массивы в Java — это структуры данных, которые позволяют хранить фиксированное количество элементов одного типа. Они предоставляют способ организовать и управлять данными в памяти, обеспечивая быстрый доступ к элементам по индексу.

1. Основные характеристики массивов в Java:

Фиксированный размер. Размер массива задается при его создании и не может быть изменен. Если вам нужно изменить размер, необходимо создать новый массив.

Однотипность. Все элементы массива должны быть одного типа (например, все целые числа, все строки и т.д.).

Индексация. Индексы массивов начинаются с 0. То есть первый элемент массива имеет индекс 0, второй — 1, и так далее до конца.

Хранение в памяти. Элементы массива хранятся в последовательных ячейках памяти, что обеспечивает быстрый доступ к элементам.

2. Объявление массивов

```
// Объявление массива целых чисел
int[] numbers;
```

```
// Альтернативный способ
int numbers[];
```

Вместо `int` может быть любой из 8 примитивов, а также другие классы, включая созданные разработчиком.

3. Инициализация массивов

```
// Инициализация массива с указанием размера
numbers = new int[5]; // Массив из 5 элементов
```

```
// Инициализация массива с заданными значениями
String[] fruits = {"яблоко", "банан", "апельсин"};
```

4. Значения по умолчанию

При создании через `new` элементы получают значения по умолчанию:

Тип элемента	Значение по умолчанию
--------------	-----------------------

int, short, byte, long	0
float, double	0.0
char	'\u0000'
boolean	false
ССЫЛОЧНЫЕ ТИПЫ	null

Пример:

```
int[] nums = new int[3];           // [0, 0, 0]
boolean[] flags = new boolean[2]; // [false, false]
String[] words = new String[2];    // [null, null]
```

5. Массивы — это объекты

Массив в Java всегда является объектом, даже если хранит примитивы. Переменная массива хранит ссылку на объект в куче.

```
int[] a = {1, 2, 3};
int[] b = a;           // b ссылается на тот же массив
b[0] = 100;
System.out.println(a[0]); // 100
```

Если массив не инициализирован, ссылка равна `null`:

```
int[] arr = null;
// arr.length — ошибка NullPointerException
```

Проверка на `null` и пустоту:

```
if (arr == null || arr.length == 0) {
    // массив отсутствует или пуст
}
```

6. Доступ к элементам массива

```
int[] numbers = {1, 2, 3, 4, 5};

// Доступ к элементу по индексу
int firstNumber = numbers[0]; // 1
int secondNumber = numbers[1]; // 2

// Изменение значения элемента
numbers[2] = 10; // Теперь массив будет {1, 2, 10, 4, 5}
```

7. Границы и исключения

Индексы допустимы от 0 до length - 1. Выход за границы приводит к ошибке – `ArrayIndexOutOfBoundsException`.

```
int[] a = {1, 2, 3};  
System.out.println(a[3]); // ошибка
```

8. Перебор элементов массива

For

```
int[] numbers = {1, 2, 3, 4, 5};  
  
for (int i = 0; i < numbers.length; i++) {  
    System.out.println(numbers[i]);  
}
```

For-each

```
for (int number : numbers) {  
    System.out.println(number);  
}
```

9. Многомерные массивы

Java поддерживает многомерные массивы, которые представляют собой массив массивов.

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9}  
};
```

Доступ к массиву

```
int value = matrix[1][2]; // 6 (второй ряд, третий столбец)  
matrix[0][1] = 10; // Теперь первый ряд будет {1, 10, 3}
```

В Java нет настоящих прямоугольных матриц — есть массив массивов. Поэтому строки могут иметь разную длину:

```
int[][] jagged = new int[3][];  
jagged[0] = new int[2];  
jagged[1] = new int[4];  
jagged[2] = new int[1];
```

Обход такого массива:

```

for (int i = 0; i < jagged.length; i++) {
    for (int j = 0; j < jagged[i].length; j++) {
        System.out.print(jagged[i][j] + " ");
    }
    System.out.println();
}

```

`matrix.length` — количество строк, `matrix[i].length` — длина конкретной строки.

10. Класс `java.util.Arrays`

Для массивов есть удобный служебный класс `Arrays`. Основные методы:

Метод	Описание
<code>sort(массив)</code>	Сортировка по возрастанию
<code>toString(массив)</code>	Преобразование в строку, например, <code>[1, 2, 3]</code>
<code>copyOf(массив, новая длина)</code>	Копия с новой длиной
<code>copyOfRange(массив, начало, конец)</code>	Копия участка
<code>fill(массив, элемент)</code>	Заполнение массива элементами
<code>equals(массив 1, массив 2)</code>	Сравнение содержимого массивов
<code>binarySearch(массив)</code>	Бинарный поиск в массиве
<code>deepToString(массив)</code>	Тоже, что и <code>toString</code> , но для многомерных массивов
<code>deepEquals(массив)</code>	Тоже, что и <code>equals</code> , но для многомерных массивов

`Arrays.toString` для многомерного массива выведет ссылки, поэтому нужен `deepToString`.

Оператор `==` сравнивает ссылки, а не содержимое, поэтому используется `equals`

```

int[] a = {1, 2};
int[] b = {1, 2};
System.out.println(a == b); // false
System.out.println(Arrays.equals(a, b)); // true

```

Копирование массивов

```
// 1. Копирование с помощью System.arraycopy
```

```
int[] original = {1, 2, 3, 4, 5};
int[] copy = new int[5];
System.arraycopy(original, 0, copy, 0, original.length);
```

```
// 2. Копирование с помощью Arrays.copyOf
int[] copy = Arrays.copyOf(original, original.length);
```

11. Алгоритмические идеи для решения задач

При решении задач на массивы полезно знать типовые приёмы. Ниже — краткий обзор идей, которые часто помогают. Конкретные задачи могут требовать комбинации нескольких подходов.

11.1. Анализ граничных случаев

Прежде чем писать код, определите:

1. что делать с пустым массивом (`length == 0`);
2. что делать с `null`;
3. что если элементов один или два;
4. что если ответ не существует;
5. какие ограничения на значения (отрицательные числа, нули, переполнение).

11.2. Сортировка целочисленного массива методом пузырька

```
int [] mas = {11, 3, 14, 16, 7};

boolean isSorted = false; // флаг, обозначающий отсортирован ли массив или нет
int buf; // переменная используется при переставлении местами элементов
while(!isSorted) { // пока массив не отсортирован выполняется цикл while
    isSorted = true; // массив отсортирован
    for (int i = 0; i < mas.length-1; i++) {
        if(mas[i] > mas[i+1]){ // если текущий элемент больше чем следующий выполняется условие
            isSorted = false; // массив не отсортирован
            buf = mas[i]; // помещаем текущий элемент во временное хранилище
            mas[i] = mas[i+1]; // в текущий элемент помещается следующий
            mas[i+1] = buf; // в следующий элемент помещается элемент из временной переменной
        }
    }
}

System.out.println(Arrays.toString(mas));
```

Код без комментариев

```
int [] mas = {11, 3, 14, 16, 7};

boolean isSorted = false;
int buf;
```

```

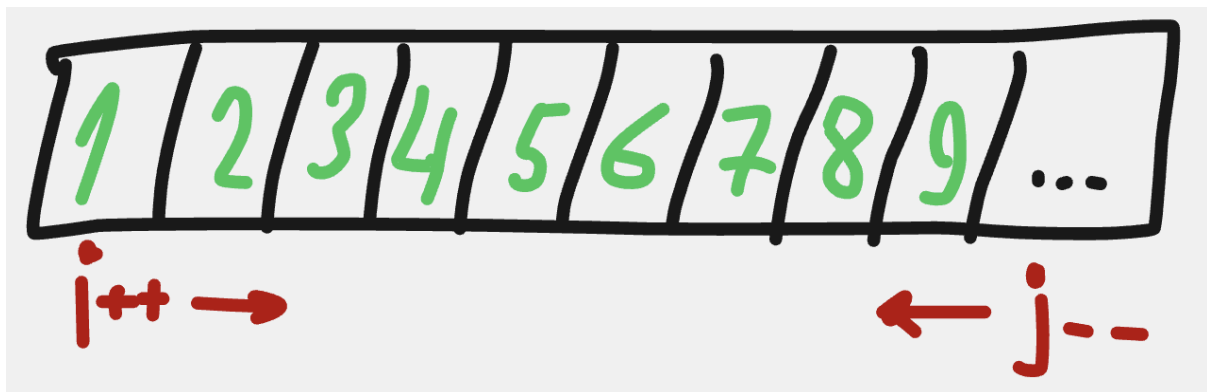
while(!isSorted) {
    isSorted = true;
    for (int i = 0; i < mas.length-1; i++) {
        if(mas[i] > mas[i+1]){
            isSorted = false;

            buf = mas[i];
            mas[i] = mas[i+1];
            mas[i+1] = buf;
        }
    }
}
System.out.println(Arrays.toString(mas));

```

11.3. Два указателя

два индекса движутся по массиву (навстречу друг другу или в одном направлении). Часто используется в отсортированных массивах.



11.4 XOR (^) помогает:

- найти уникальный элемент среди парных;
- поменять значения без временной переменной;
- проверить четность.

XOR (в Java, C, C++ и многих других языках — оператор ^) — это побитовое исключающее ИЛИ.

Он работает с отдельными битами чисел по правилу:

$$\begin{aligned}
 0 \wedge 0 &= 0 \\
 0 \wedge 1 &= 1 \\
 1 \wedge 0 &= 1 \\
 1 \wedge 1 &= 0
 \end{aligned}$$

Иначе говоря, результат равен 1, только если биты различаются.

Основные свойства XOR:

$a \oplus a = 0$ — число, сложенное XOR само с собой, даёт ноль.

$a \oplus 0 = a$ — ноль не меняет число.

$a \oplus b = b \oplus a$ — коммутативность.

$(a \oplus b) \oplus c = a \oplus (b \oplus c)$ — ассоциативность.

$a \oplus b \oplus a = b$ — полезно для поиска и обмена.

Если в массиве каждый элемент встречается дважды, кроме одного, то XOR всех элементов даст этот уникальный элемент. Парные числа взаимно уничтожаются: $x \oplus x = 0$. Остаётся только то, что не имеет пары.

Пример:

```
int[] nums = {4, 1, 2, 1, 2};
int unique = 0;
for (int x : nums) {
    unique ^= x;
}
// unique = 4
```

По шагам:

$0 \oplus 4 = 4$

$4 \oplus 1 = 5$

$5 \oplus 2 = 7$

$7 \oplus 1 = 6$

$6 \oplus 2 = 4$

Бит чётности (parity bit)

XOR позволяет вычислить чётность количества единичных битов.

Если XOR всех битов числа равен 0, то единиц чётное число. Если 1 — нечётное.

Пример для числа 5 (0101):

$0 \oplus 1 \oplus 0 \oplus 1 = 0$

Единиц две — чётное количество.

Это используется в помехоустойчивом кодировании, контрольных суммах, криптографии.

Переключение чётности

XOR с 1 инвертирует младший бит:

$x \oplus 1$

Если x было чётным, станет нечётным, и наоборот:

$4 \oplus 1 = 5$

$5 \oplus 1 = 4$

Выделение младшего единичного бита (8-я задача)

Выражение $x \& -x$ (или $x \& \sim(x - 1)$, или `Integer.lowestOneBit(x)`) оставляет только самый правый единичный бит числа x , обнуляя все остальные.

Отрицательные числа в Java хранятся в дополнительном коде: $-x = \sim x + 1$.

Инвертирование x превращает биты ниже младшей единицы в 1, а саму единицу — в 0. Прибавление 1 возвращает единицу на прежнее место и обнуляет всё, что было ниже. В результате у $-x$ совпадает с x только младший единичный бит, а все остальные биты различаются. Операция $\&$ оставляет ровно этот бит.

```
x    = 6 = 0000 0110
-x   =    1111 1010
x & -x = 0000 0010 = 2
```

Когда применять:

- нужно найти любой различающийся бит между двумя числами;
- нужно разделить числа на две группы по конкретному биту;
- нужно изолировать младшую единицу при работе с битовыми масками;
- в задачах на XOR, где результат $a \wedge b \neq 0$ и нужно понять, чем a и b отличаются.

Типовой сценарий в массивах:

- Получить $\text{xorAll} = a \wedge b$ (после XOR всех элементов).
- Найти $\text{diffBit} = \text{xorAll} \& -\text{xorAll}$.
- Разделить все числа на две группы: у кого diffBit стоит — в одну, у кого нет — в другую.
- XOR внутри каждой группы даёт a и b по отдельности.

11.5. Паддинг нулями (3-я задача)

Чтобы не проверять границы вручную, можно расширить матрицу на n нулей со всех сторон:

- новая матрица размера $(N + 2n) \times (N + 2n)$;
- исходные данные копируются в центр;
- для каждой исходной клетки (i, j) её центр в расширенной матрице — $(i + n, j + n)$;
- квадрат $(2n+1) \times (2n+1)$ вокруг центра всегда полностью помещается в расширенную матрицу;
- сумма берётся через префиксные суммы по расширенной матрице без обрезаний.

Плюс: код становится однородным, нет условий на границы.

Задания

Задания выбирать: номер в списке
группы mod 10
 $19 \bmod 10 = 9$

Задача 0: Сжатие массива символов

Дан массив символов `char[] chars`, содержащий последовательности одинаковых символов.

Сожмите его, заменяя каждую последовательность на символ и число его повторений (если повторений больше 1).

Число записывайте по одной цифре. Верните новую длину массива.

Измените массив на месте. Порядок групп сохраните.

Пример:

`chars = {'a','a','b','b','c','c','c'}`

Результат: `chars = {'a','2','b','2','c','3', ...}`, новая длина = 6

Задача 1: Поворот матрицы

Напишите алгоритм, который поворачивает квадратную матрицу на 90 градусов по часовой стрелке. Измените матрицу на месте (без использования дополнительной памяти для новой матрицы). Обработайте случай, когда матрица не квадратная.

Задача 2: Проверка на палиндром

Проверьте, является ли массив целых чисел палиндромом.

Массив называется палиндромом, если он читается одинаково слева направо и справа налево.

Нельзя использовать дополнительный массив. Постарайтесь обойтись без сортировки.

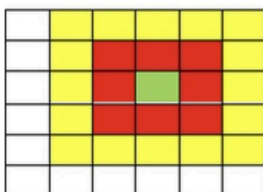
Пример:

`nums = [1, 2, 3, 2, 1] → true`

`nums = [1, 2, 3, 4] → false`

Задача 3: Замена элементов матрицы на среднее арифметическое

Дана квадратная матрица и число n . Замените каждый элемент матрицы на среднее арифметическое элементов квадрата размера $2n + 1$ с центром в данном элементе.



Например, для элемента, выделенного зеленым, при $n = 1$ должно вычисляться среднее арифметическое элементов красного и зеленого цвета, а для $n = 2$ —

среднее арифметическое желтых, красных и зелёного элементов. Если квадрат выходит за пределы матрицы, считать недостающие элементы равными нулю:

		0	0	0
				0
				0

Задача 4: Проверка сбалансированности скобок

Напишите алгоритм, который проверяет, является ли строка с скобками сбалансированной. Поддерживайте круглые, квадратные и фигурные скобки.

1. Сбалансированные скобки:

- `()`
- `{[]}`
- `((a + b) * (c - d))`

2. Несбалансированные скобки:

- `(` (открывающая скобка без закрывающей)
- `)` (закрывающая скобка без соответствующей открывающей)
- `{[]}` (вложение скобок неправильное)

Задача 5: Переместить нули в конец

Дан массив целых чисел `nums`. Переместите все нули в конец массива, сохранив относительный порядок ненулевых элементов.

Измените массив на месте, не создавая новый. Постарайтесь выполнить за один проход.

Пример:

`nums = [0, 1, 0, 3, 12]`

Результат: `[1, 3, 12, 0, 0]`

Задача 6: Сортировка массива по четным и нечетным индексам

Напишите алгоритм, который сортирует массив, оставляя четные элементы на четных индексах, а нечетные — на нечетных. Сортируйте элементы по возрастанию, сохраняя их индексы.

Задача 7: Циклический сдвиг массива вправо

Дан массив целых чисел `nums` и целое число `k`. Сдвиньте все элементы массива вправо на `k` позиций циклически.

Измените массив на месте. `k` может быть больше длины массива.

Пример:

`nums = [1, 2, 3, 4, 5], k = 2`

Результат: `[4, 5, 1, 2, 3]`

Задача 8: Два уникальных элемента

Дан массив целых чисел `nums`, в котором каждое число встречается ровно два раза, кроме двух чисел, которые встречаются по одному разу.

Найдите эти два уникальных числа.

Пример:

`nums = [1, 2, 1, 3, 2, 5]`

Уникальные числа: 3 и 5.

Результат: [3, 5] (порядок не важен).

Задача 9: Шифр Цезаря

Дан массив символов `char[] text`, содержащий только заглавные латинские буквы 'A'..'Z', и целое число `shift`.

Зашифруйте текст, сдвинув каждую букву на `shift` позиций по кругу (например, 'Z' при сдвиге 1 переходит в 'A').

Измените массив на месте.

Пример:

`text = {'H', 'E', 'L', 'L', 'O'}, shift = 3`

Результат: {'K', 'H', 'O', 'O', 'R'}